Cybersecurity Threat Assessment and Mitigation Plan: CI/CD Pipeline Compromise

Scenario

MagRutTech Ltd is a mid-sized software company serving enterprise clients in the financial sector. In early 2025, the company discovers that its CI/CD pipeline has been compromised. A threat actor injected malicious code directly into the build process, and the code propagated to production artefacts before deployment. The breach went undetected for six weeks.

Threat Assessment

Three vulnerabilities enabled the attack. First, the pipeline retrieved packages from public repositories without verifying their integrity. An attacker published a malicious package under a name matching an internal dependency — a technique known as dependency confusion. The build system fetched it without challenge. Second, deployment credentials were stored as plaintext environment variables in pipeline configuration files, which were visible to anyone with repository read access. Third, there was no monitoring capable of detecting unusual build behaviour. Network calls originating from within the build environment went unnoticed for weeks.

This pattern is not hypothetical. The SolarWinds compromise demonstrated precisely how tampered build artefacts can reach thousands of downstream organisations before anyone notices (CISA, 2021). MagRutTech's situation follows the same logic on a smaller scale.

Mitigation Plan

Mitigation needs to operate at three levels.

At the pipeline level, all dependencies must be pinned to verified versions and validated against a Software Bill of Materials (SBOM). Build artefacts should be cryptographically signed, and tools like Sigstore make this practical without significant overhead. The SLSA framework offers a graduated supply chain security model that is well-suited to organisations of MagRutTech's size (SLSA, 2023).

At the infrastructure level, plaintext credential storage must go. Dedicated secrets management tooling should replace environment variables entirely, with access governed by least-privilege principles. This directly addresses the access control requirements outlined in ISO/IEC 27001:2022. Kim et al. (2016) make the point clearly: security cannot be bolted on at the point of deployment. It must be embedded throughout the delivery pipeline. Credential hygiene is a straightforward place to start.

At the detection level, pipeline monitoring should flag irregular build behaviour and unexpected outbound connections as they happen. The NIST Secure Software Development Framework provides structured, practical guidance for embedding these controls across the development lifecycle (NIST, 2022).

Ethical and Legal Considerations

The six-week detection gap creates immediate legal exposure. Under GDPR Article 33, organisations must notify the relevant supervisory authority within 72 hours of becoming aware of a personal data breach (European Parliament, 2016). A six-week gap makes it impossible to meet that obligation retrospectively.

The ethical dimension extends further. MagRutTech's financial sector clients received compromised software without their knowledge. That creates a duty of transparency that goes beyond regulatory requirements. Shostack (2014) argues that threat modelling must account for harm to downstream parties, not only the organisation directly breached — a point that applies directly here.

Conclusion

A compromised CI/CD pipeline is a credible and underreported threat vector. At MagRutTech, three interconnected weaknesses created the conditions for a serious breach. Addressing them requires coordinated controls at the pipeline, infrastructure, and detection levels. The legal and ethical obligations that follow a breach of this kind reinforce what the technical case already makes clear: supply chain security is not optional.

References

CISA (2021) *Alert (AA20-352A): Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations*. Available at: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a [Accessed: 26 March 2026].

European Parliament (2016) *Regulation (EU) 2016/679 of the European Parliament and of the Council*, Article 33. Official Journal of the European Union, L 119, pp. 1–88. Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679 [Accessed: 26 March 2026].

Kim, G., Humble, J., Debois, P. and Willis, J. (2016) *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland: IT Revolution Press.

NIST (2022) *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities* (SP 800-218). Available at: https://doi.org/10.6028/NIST.SP.800-218 [Accessed: 26 March 2026].

Shostack, A. (2014) *Threat Modeling: Designing for Security*. Indianapolis: Wiley.

SLSA (2023) *Supply-chain Levels for Software Artifacts*. Available at: https://slsa.dev [Accessed: 26 March 2026].